

AIML SESSION 07 ASSIGNMENT SUBMISSION

Name of Student : Vaibhav Vijay Kate

Domain/Subject : AIML (Artificial Intelligence & Machine Learning)

College : Government Polytechnic College Washim

Branch : Information Technology

Session : 07

Q1

Q1. Creating Tuples)

Write a program to create tuples using different methods:

- a) A tuple with 5 numbers using parentheses.
- b) A mixed tuple containing integer, string, float, and boolean.
- c) An empty tuple using two different ways.
- d) A single-element tuple with the value 99.

Print all tuples and their types. Add comments explaining the rules for single-element tuples.

Program Code:

```
# A tuple with 5 numbers using parentheses.
```

```
tup1 = (1,2,3,4,5)
```

```
# A mixed tuple containing integer, string, float, and boolean.
```

```
tup2 = (1,"Vaibhav",1.3,True)
```

```
# An empty tuple using two different ways.
```

```
# Method 1
```

```
tup3 = ()
```

```
# Method 2
```

```
tup4 = tuple()
```

```
# A single-element tuple with the value 99.
```

```
tup5 = (99,) # Single-element tuple (comma is required)
```

```
# Print all tuples and their types. Add comments explaining the rules for single element tuples.
```

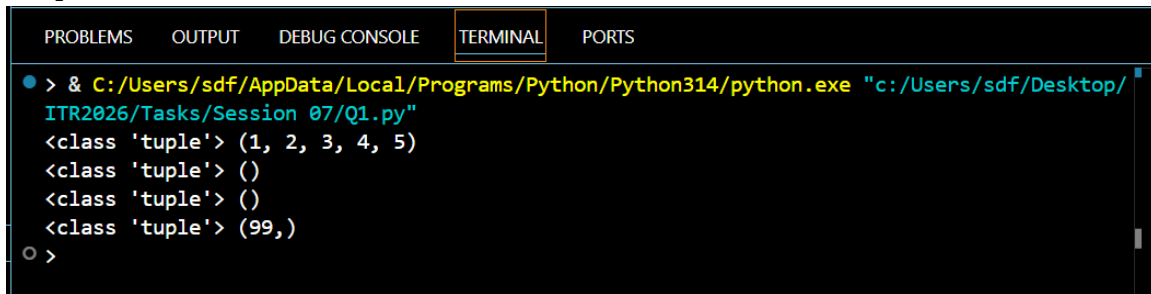
```
print(type(tup1),tup1)
```

```
print(type(tup2),tup3)
```

```
print(type(tup4),tup4)
```

```
print(type(tup5),tup5)
```

Output Screenshot:



The screenshot shows a terminal window with a dark background and light-colored text. At the top, there is a tab bar with five tabs: "PROBLEMS", "OUTPUT", "DEBUG CONSOLE", "TERMINAL" (which is selected and highlighted with a blue border), and "PORTS". Below the tab bar, the terminal displays the following text:

```
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 07/Q1.py"
<class 'tuple'> (1, 2, 3, 4, 5)
<class 'tuple'> ()
<class 'tuple'> ()
<class 'tuple'> (99,)
○ >
```

The command prompt shows the execution of a Python script. The output consists of four lines, each showing a tuple object. The first tuple contains the numbers 1, 2, 3, 4, and 5. The second and third tuples are empty. The fourth tuple contains the number 99. The prompt at the bottom is a small circle followed by a greater-than sign.

Q2

Q2. Tuple Packing)

Create a tuple without using parentheses (tuple packing) to store: your name, age, and city.

Print the tuple and its type.

Then unpack the tuple into three separate variables and print them individually.

Program Code:

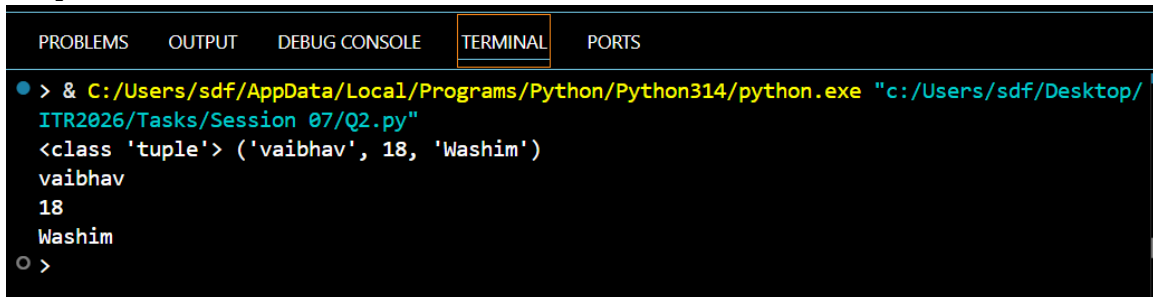
```
# Tuple Packing
tuple_Data = "vaibhav", 18, "Washim"

print(type(tuple_Data), tuple_Data)

# Tuple Unpacking
name, age, city = tuple_Data

print(name)
print(age)
print(city)
```

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 07/Q2.py"
<class 'tuple'> ('vaibhav', 18, 'Washim')
vaibhav
18
Washim
>
```

Q3

Q3. Indexing and Negative Indexing)

Create the tuple: colors = ('red', 'green', 'blue', 'yellow', 'purple', 'orange')

Print:

First element

Third element

Last element (using negative index)

Second last element (using negative index)

Take an index number as input from the user and print the element at that index.

Program Code:

```
# Creating a Tuple
```

```
colors = ('red', 'green', 'blue', 'yellow', 'purple', 'orange')
```

```
# First element
```

```
print(colors[0])
```

```
# Third element
```

```
print(colors[2])
```

```
# Last element (using negative index)
```

```
print(colors[-1])
```

```
# Second last element (using negative index)
```

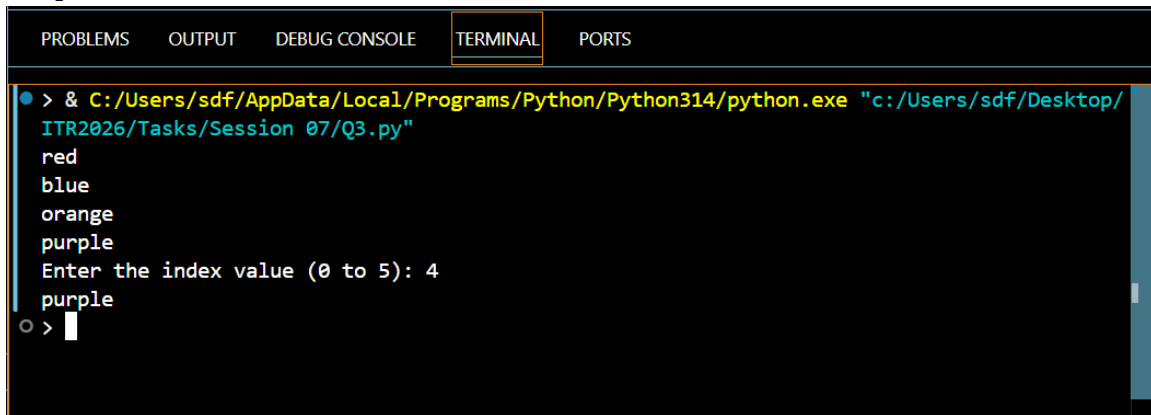
```
print(colors[-2])
```

```
# Take an index number as input from the user and print the element at that index.
```

```
a = int(input("Enter the index value (0 to 5): "))
```

```
print(colors[a])
```

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 07/Q3.py"
red
blue
orange
purple
Enter the index value (0 to 5): 4
purple
>
```

Q4

Q4. Slicing Tuples)

Given the tuple: numbers = 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Using slicing, print:

a) Elements from index 2 to 7

b) First 5 elements

Session 07 (AIML) - Assignment Questions 1

c) Last 4 elements

d) Every second element

e) The entire tuple in reverse order

Add comments explaining the slicing syntax.

Program Code:

```
# Given the tuple: numbers = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

```
# Slicing Syntax :
```

```
# sequence [start : Stop : step]
```

```
numbers = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

```
# Elements from index 2 to 7
```

```
print(numbers[2:7])
```

```
# First 5 elements
```

```
print(numbers[0:5])
```

```
# Last 4 elements
```

```
print(numbers[7:11])
```

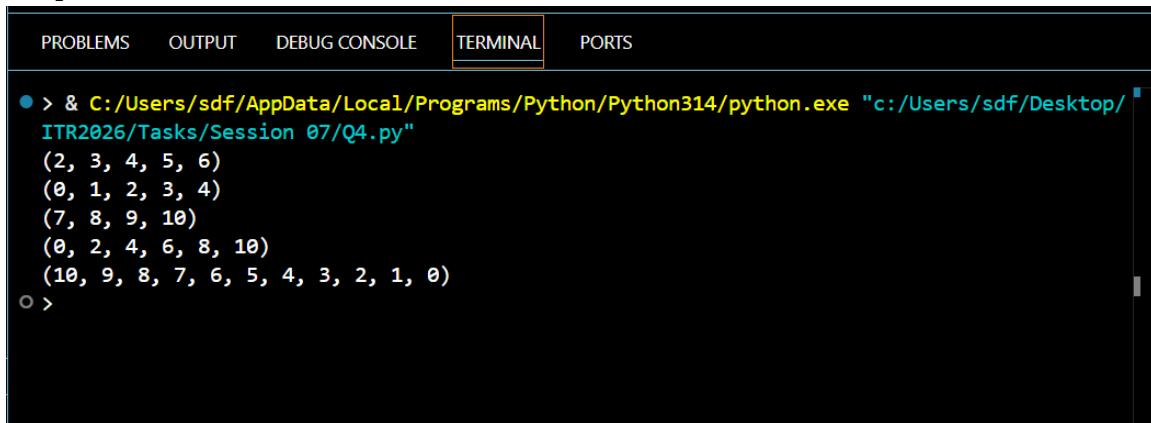
```
# Every second element
```

```
print(numbers[::2])
```

```
# The entire tuple in reverse order
```

```
print(numbers[::-1])
```

Output Screenshot:



The screenshot shows a code editor interface with a terminal window at the bottom. The terminal has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (which is active), and PORTS. The terminal displays the command to run a Python script and its output.

```
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 07/Q4.py"
(2, 3, 4, 5, 6)
(0, 1, 2, 3, 4)
(7, 8, 9, 10)
(0, 2, 4, 6, 8, 10)
(10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0)
>
```


Q5

Q5. Nested Tuples)

Create a nested tuple to represent a 3x3 matrix:

matrix = 1, 2, 3, 4, 5, 6, 7, 8, 9

Print:

The first row

The element at second row, third column

The element at third row, first column

Program Code:

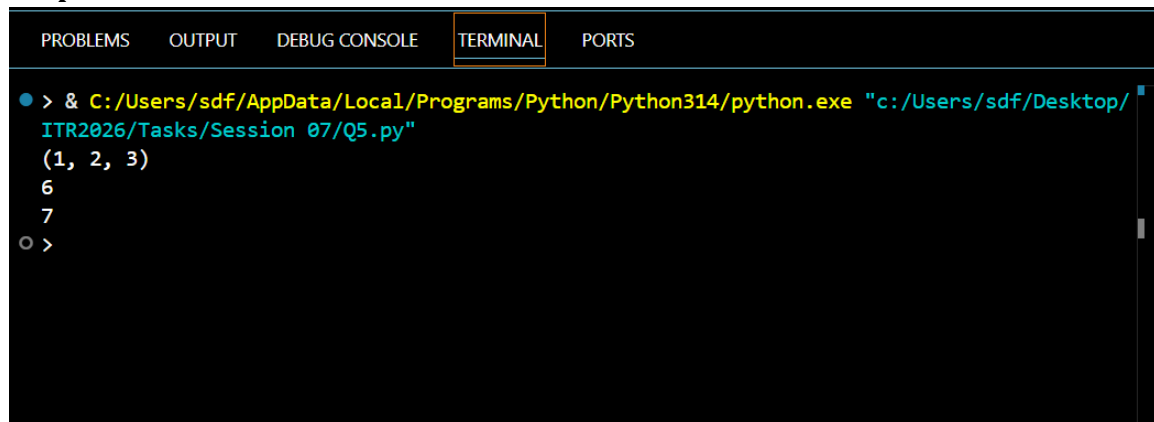
```
matrix = ((1, 2, 3),  
          (4, 5, 6),  
          (7, 8, 9))
```

```
# The first row  
print(matrix[0])
```

```
# The element at second row, third column  
print(matrix[1][2])
```

```
# The element at third row, first column  
print(matrix[2][0])
```

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
● > & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/  
ITR2026/Tasks/Session 07/Q5.py"  
(1, 2, 3)  
6  
7  
○ >
```

Q6

Q6. Iterating and Unpacking)

Create a tuple: student = ('Rahul', 20, 'Computer Science', 'A')

a) Iterate through the tuple using a for loop and print each item.

b) Unpack the tuple into four variables (name, age, branch, grade) and print them with descriptive messages.

Program Code:

```
student = ('Rahul', 20, 'Computer Science', 'A')
```

```
# a) Iterating
```

```
print("Tuple Elements:")
```

```
for item in student:
```

```
    print(item)
```

```
# b) Unpacking
```

```
name, age, branch, grade = student
```

```
print("\nStudent Details:")
```

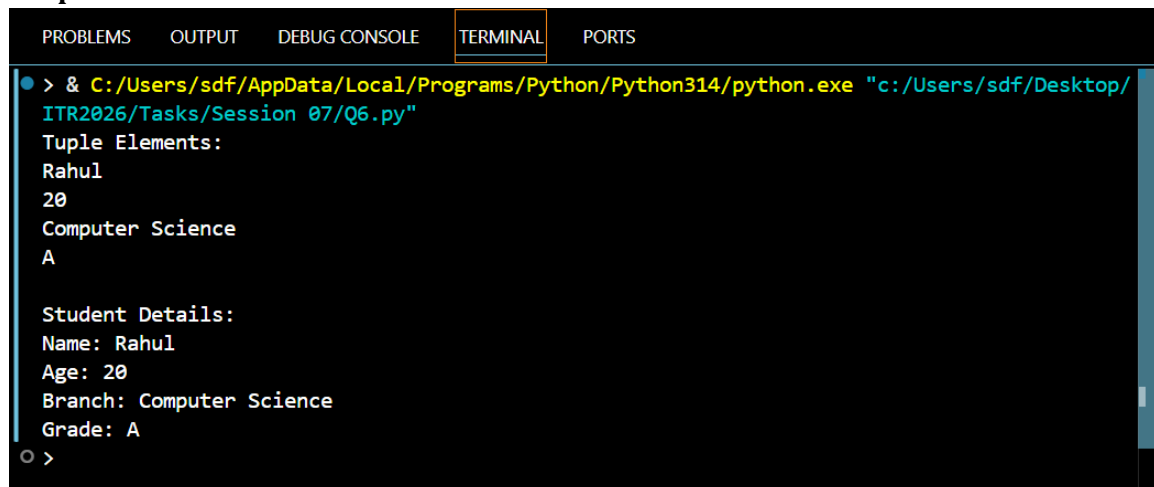
```
print("Name:", name)
```

```
print("Age:", age)
```

```
print("Branch:", branch)
```

```
print("Grade:", grade)
```

Output Screenshot:



```
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 07/Q6.py"
Tuple Elements:
Rahul
20
Computer Science
A

Student Details:
Name: Rahul
Age: 20
Branch: Computer Science
Grade: A
>
```

Q7

Q7. (count() Method)

Create the tuple: grades = ('A', 'B', 'A', 'C', 'A', 'B', 'D', 'A', 'B')

Count and print how many times 'A' appears.

Count and print how many times 'B' appears.

Take a grade as input from the user and print how many times it appears.

Program Code:

```
# Creating a Tuple
```

```
grades = ('A', 'B', 'A', 'C', 'A', 'B', 'D', 'A', 'B')
```

```
# Count and print how many times 'A' appears.
```

```
print("A Appears :",grades.count('A'),"Times")
```

```
# Count and print how many times 'B' appears.
```

```
print("B is Appears :",grades.count('B'),"Times")
```

```
# Take a grade as input from the user and print how many times it appears.
```

```
grade = input("Enter the grade to search: ")
```

```
count = grades.count(grade)
```

```
print(f'{grade} appears {count} time(s) in the tuple.')
```

Output Screenshot:

```
Python: Q7 + - [ ] [ ] ... [ ] [ ] [ ] [ ]
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session
07/Q7.py"
A Appears : 4 Times
B is Appears : 3 Times
Enter the grade to search: C
C appears 1 time(s) in the tuple.
>
```

Q8

Q8. (index() Method)

Create the tuple: fruits = ('apple', 'banana', 'cherry', 'banana', 'mango', 'apple')

Find and print the first index of 'banana'.

Find and print the first index of 'banana' starting the search from index 2.

Use try-except to safely find the index of 'kiwi' and print "Not found" if it doesn't exist.

Session 07 (AIML) - Assignment Questions 2

Program Code:

```
# Creating a Tuple
```

```
fruits = ('apple', 'banana', 'cherry', 'banana', 'mango', 'apple')
```

```
# Find and print the first index of 'banana'.
```

```
print("Frist Index Of a banana :", fruits.index('banana'))
```

```
# Find and print the first index of 'banana' starting the search from index 2.
```

```
print("Frist Index Of a banana :", fruits.index('banana', 2))
```

```
# Use try-except to safely find the index of 'kiwi' and print "Not found" if it
```

```
fruits = ('apple', 'banana', 'mango', 'orange')
```

```
try:
```

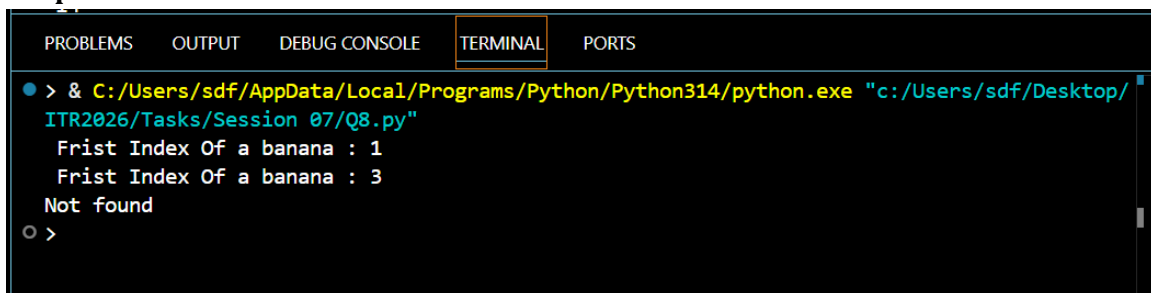
```
    index = fruits.index('kiwi')
```

```
    print("Index of kiwi:", index)
```

```
except ValueError:
```

```
    print("Not found")
```

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 07/Q8.py"
Frist Index Of a banana : 1
Frist Index Of a banana : 3
Not found
>
```

Q9

Q9. Immutability of Tuples)

Create a tuple: coordinates = 10, 20

Demonstrate that tuples are immutable by trying to:

Change the first element

Add a new element using append()

Explain what errors occur (in comments). Then show the correct way to modify data by converting the tuple to a list, making changes, and converting back to a tuple.

Program Code:

```
# (Immutability of Tuples)
```

```
# Create a tuple: coordinates = (10, 20)
```

```
# Demonstrate that tuples are immutable by trying to:
```

```
# Change the first element
```

```
# Add a new element using append()
```

```
# Explain what errors occur (in comments). Then show the correct way to modify
```

```
# data by converting the tuple to a list, making changes, and converting back to a
```

```
# tuple.
```

```
# Create a tuple
```

```
coordinates = (10, 20)
```

```
print("Original Tuple:", coordinates)
```

```
# Attempt 1: Change the first element
```

```
#coordinates[0] = 100
```

```
# Original Tuple: (10, 20)
```

```
# Traceback (most recent call last):
```

```
# File "c:\Users\sdf\Desktop\ITR2026\Tasks\Session 07\Q9.py", line 21, in <module>
```

```
#   coordinates[0] = 100
```

```
# TypeError: 'tuple' object does not support item assignment
```

```
# Reason:
```

```
# Tuples are immutable, so their elements cannot be changed after creation.
```

```
# Attempt 2: Add a new element using append()
```

```
# coordinates.append(30)
```

```
# Error:
# AttributeError: 'tuple' object has no attribute 'append'
# Reason:
# Tuples do not have methods like append(), insert(), or remove().
```

```
# Correct Way: Convert tuple to list, modify, then convert back
```

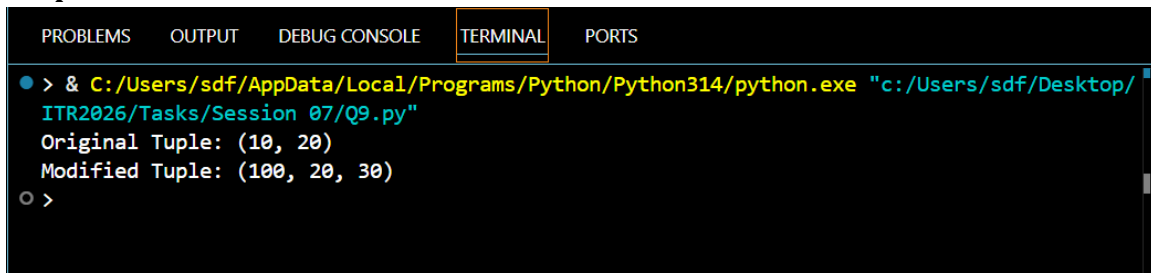
```
# Convert tuple to list
coordinates_list = list(coordinates)
```

```
# Modify data
coordinates_list[0] = 100
coordinates_list.append(30)
```

```
# Convert back to tuple
coordinates = tuple(coordinates_list)
```

```
print("Modified Tuple:", coordinates)
```

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 07/Q9.py"
Original Tuple: (10, 20)
Modified Tuple: (100, 20, 30)
>
```

Q10

Q10. Mini Project - Combined Concepts)

Create a Student Record Program using tuples:

Take input for name, roll number, 3 subject marks.

Store all information in a tuple (use packing).

Print the complete record.

Unpack the tuple and print each value with proper labels.

Use count() to check how many times a particular mark appears (if any).

Create a nested tuple to store multiple student records (at least 2 students) and access specific values.

Add meaningful comments and use proper formatting.

Session 07 (AIML) - Assignment Questions 3

Program Code:

```
# Q10. (Mini Project - Combined Concepts)
```

```
# Student Record Program using Tuples
```

```
# Take input from the user
```

```
name = input("Enter Student Name: ")
```

```
roll_no = int(input("Enter Roll Number: "))
```

```
mark1 = int(input("Enter Subject 1 Marks: "))
```

```
mark2 = int(input("Enter Subject 2 Marks: "))
```

```
mark3 = int(input("Enter Subject 3 Marks: "))
```

```
# Store all information in a tuple (Packing)
```

```
student = (name, roll_no, mark1, mark2, mark3)
```

```
# Print complete record
```

```
print("\n===== Student Record =====")
```

```
print(student)
```

```
# Unpack the tuple
```

```
name, roll_no, mark1, mark2, mark3 = student
```

```
# Print values with labels
```

```
print("\n===== Student Details =====")
```

```
print("Name    :", name)
```

```
print("Roll No  :", roll_no)
```

```

print("Subject 1 :", mark1)
print("Subject 2 :", mark2)
print("Subject 3 :", mark3)

# Count how many times a mark appears
search_mark = int(input("\nEnter a mark to count: "))

count_mark = student.count(search_mark)

print(f"{search_mark} appears {count_mark} time(s) in the record.")

# Create Nested Tuple (Multiple Students)
student1 = student

student2 = ("Rahul",102,85,90,88)

students = (student1, student2)

# Print Nested Tuple
print("\n===== All Student Records =====")
print(students)

# Access Specific Values
print("\n===== Accessing Specific Values =====")

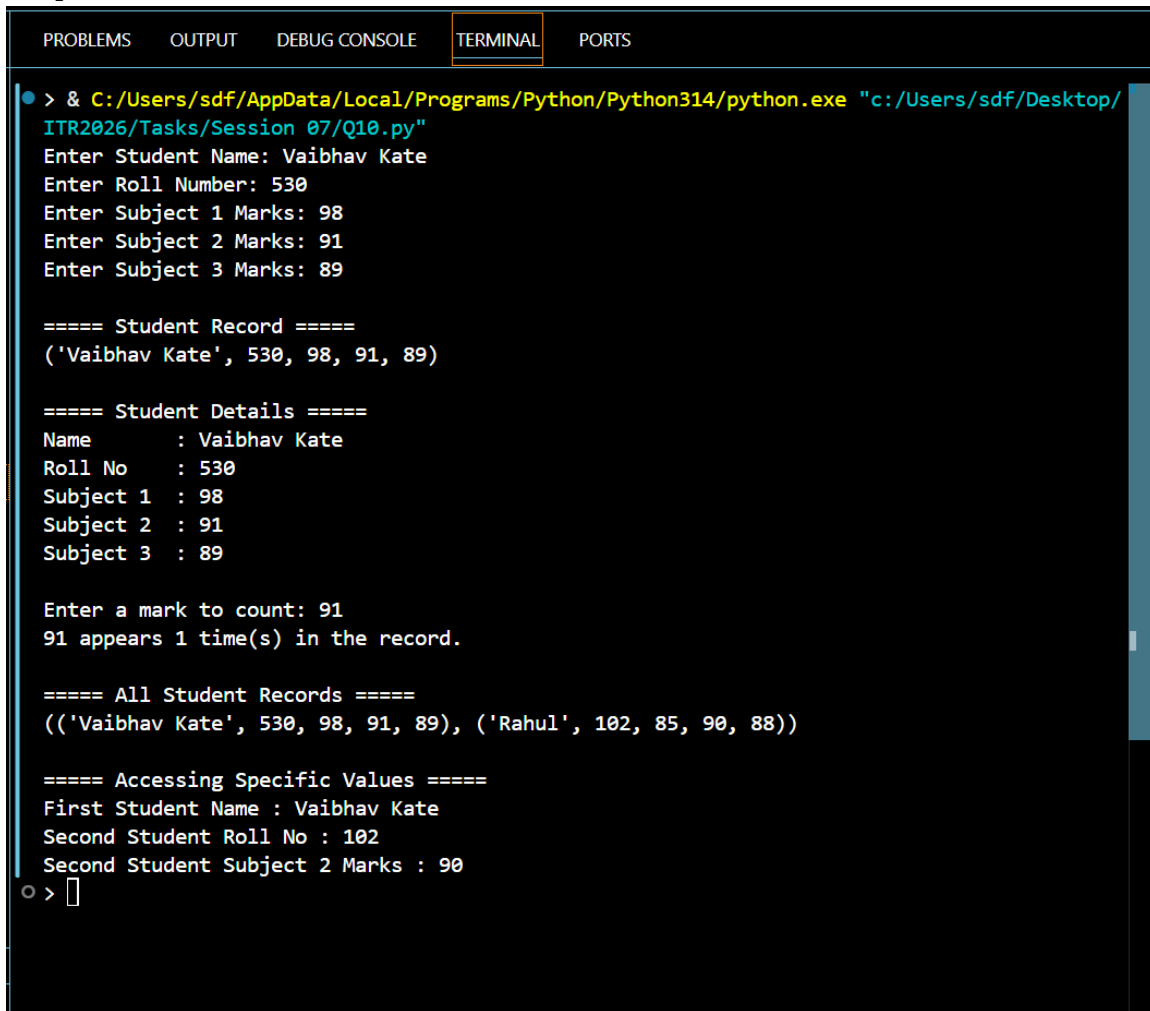
# First student's name
print("First Student Name :", students[0][0])

# Second student's roll number
print("Second Student Roll No :", students[1][1])

# Second student's Subject 2 marks
print("Second Student Subject 2 Marks :", students[1][3])

```


Output Screenshot:



```
> & C:/Users/sdf/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sdf/Desktop/ITR2026/Tasks/Session 07/Q10.py"
Enter Student Name: Vaibhav Kate
Enter Roll Number: 530
Enter Subject 1 Marks: 98
Enter Subject 2 Marks: 91
Enter Subject 3 Marks: 89

===== Student Record =====
('Vaibhav Kate', 530, 98, 91, 89)

===== Student Details =====
Name      : Vaibhav Kate
Roll No   : 530
Subject 1 : 98
Subject 2 : 91
Subject 3 : 89

Enter a mark to count: 91
91 appears 1 time(s) in the record.

===== All Student Records =====
(('Vaibhav Kate', 530, 98, 91, 89), ('Rahul', 102, 85, 90, 88))

===== Accessing Specific Values =====
First Student Name : Vaibhav Kate
Second Student Roll No : 102
Second Student Subject 2 Marks : 90
> 
```